

HÁZI FELADAT

Programozás alapjai 3.

Feladatválasztás/feladatspecifikáció Amőba játék

Kálmán Patrik
XG5YQ1

2025. december 01.

TARTALOM

1. Feladat	2
2. Feladatspecifikáció.....	2
3. Terv	3
3.1 Objektum terv.....	3
4. Megvalósítás.....	10
4.1 Osztályok bemutatása.....	10
4.2 Osztályok főbb metódusainak bemutatása	11
4.3 Fájlkezelés	29
5. Tesztelés	30
6. Felhasználói kézikönyv	33
7. Források.....	37

1. Feladat

Amőba játékleírása

Az amőba egy klasszikus logikai játék, amelyben a stratégiai gondolkodás kulcsfontosságú.

A játékot két játékos játssza egymás ellen: lehet két ember, vagy egy ember és számítógép. Mindkét játékos egy-egy jelet használ: az egyik a kört (O), a másik az ikszet (X).

A cél az, hogy a játékos saját jeléből ötöt egymás mellé helyezzen el vízszintesen, függőlegesen vagy átlósan a játéktáblán. Aki ezt először megteszi, megnyeri a játékot.

2. Feladatspecifikáció

A feladat egy egyszerű, klasszikus amőba játék megtervezése és megvalósítása Java nyelven, objektumorientált programozás és grafikus felhasználói felület (GUI) alkalmazásával. A játéktábla (pálya) **15×15 mezőből** áll.

Főmenü

A program indításakor egy menü jelenik meg, amely három menüpontot tartalmaz:

- **Beállítások:** a játék paramétereinek testreszabása
 - az alakzat beállítása (kör = A játékos; iksz = B játékos)
 - a játékmódja (1 = gép ellen (bot); 2 = két játékos)
 - ha gép ellen játszunk
 - a kezdés sorrendje
 - a nehézségi szint (könnyű, közepes, nehéz)
- **Játék (új játék):** új játék indítása
- **Kilépés:** a játékból való kilépés

A játék menete:

Az „Új játék” gomb megnyomása után megjelenik a játéktábla, és a felső sávban látható, hogy ki kezdi a játékot. A játékos a pálya egyik üres mezőjére kattintva helyezheti el saját alakzatunkat. A játék során a cél, hogy a saját jeleinkből öt darabot egymás mellé helyezzünk el vízszintesen, függőlegesen vagy átlósan.

A program ellenőrzi a játék állapotát (sor, oszlop, átló), ha valamelyik játékos nyer vagy győz, akkor a megadott a nyerő pontok szint váltanak.

Ritkább esetekben van olyan amikor a játék kimenete döntetlen, ez abban az esetben fordulhat elő, mikor egy játékos sem tudta kirakni a saját jeléből egymás mellé az öt darabot.

Játék vége

A játék befejezése után a program visszatér a főmenübe, ahol a játékos új játékot indíthat, módosíthatja a beállításokat, vagy kiléphet az alkalmazásból.

3. Terv

A feladat egy grafikus felületű Amőba játék megtervezését és megvalósítását igényli Java nyelven, Swing grafikus eszköztárral. A program támogatja az egy- és kétjátékos módot, többféle nehézségi szinttel, valamint egy konfigurációs felületet biztosít a játékos beállítások testreszabásához.

3.1 Objektum terv

Az Amőba játék több egymással együttműködő osztályból álló objektumrendszerrel valósul meg. Az osztályrendszer feladata a játék logikájának, a grafikus felület működésének és a bot mesterséges intelligenciájának kezelése.

Az osztálystruktúra főbb részei:

Menu osztály

A **Menu osztály** a program grafikus felületének kezdőképernyőjét valósítja meg. A játék indításakor ez az osztály felel a főmenü megjelenítéséért és a felhasználó navigációjáért. A Menu a **JFrame** osztályból öröklődik, így önálló alkalmazásablakként működik, amely tartalmazza a főmenü vezérlőelemeit (gombok, felirat, háttér).

Adattagok:

- **JLabel felirat:** A menü felső részén megjelenő cím ("**Amőba játék**").
- **JButton play:** Új játék indításáért felelős gomb.
- **JButton settings:** A beállítási ablakot megnyitó gomb.
- **JButton quit:** A program bezárását végző gomb.
- **JPanel gombPanel:** A gombok függőleges elrendezésére használt panel.

Metódusok

- **Menu()** – Konstruktor

Eseménykezelők:

- **Quit_Click:** A kilépés gomb kiválasztását kezeli.
- **Settings_Click:** A beállítások gomb kiválasztását kezeli.
- **Play_Click:** Az új játék gomb kiválasztását kezeli.

Settings osztály

A Settings osztály a játék beállítási felületét valósítja meg. Ez az ablak lehetővé teszi a felhasználó számára a játék paramétereinek testreszabását, például a játékosok számát, a nehézségi szintet, a kezdő játékos kiválasztását, valamint a játékosok jelének (X vagy O) meghatározását. A Settings a JFrame osztályból öröklődik, így önálló alkalmazásablakként működik.

Adattagok

- **JLabel felirat:** a beállítási ablak felső részén megjelenő cím („Beállítások”).
- **DifficultyPanel difficultyPanel:** a játék nehézségi szintjét kezelő panel.
- **PlayerNumberPanel playerNumberPanel:** a játékosok számát kezelő panel.
- **StartingPlayerPanel startingPlayerPanel:** a kezdő játékost kiválasztó panel.
- **SymbolSelectionPanel symbolSelectionPanel:** a játékosok jelének kiválasztását lehetővé tevő panel.

Metódusok

- **Settings():** Konstruktor

DifficultyPanel osztály

A DifficultyPanel osztály a beállítási ablak egyik alpanelje, amely lehetővé teszi a játék nehézségi szintjének kiválasztását. A panel a JPanel osztályból származik, így önálló komponensként működik a GUI-ban.

Adattagok:

- **JButton kezdő:** Könnyű szint gomb.
- **JButton közepes:** Közepes szint gomb.
- **JButton nehéz:** Nehéz szint gomb.

Metódusok:

- **DifficultyPanel():** Konstruktor
- **updateGombok():** void

Eseménykezelők:

- **Konnyu_Click:** A könnyű gomb kiválasztását kezeli.
- **Kozepes_Click:** A közepes gomb kiválasztását kezeli.
- **Nehéz_Click:** A nehéz gomb kiválasztását kezeli.

PlayerNumberPanel osztály

A PlayerNumberPanel osztály a beállítási ablak egyik alpanelje, amely a játékosok számának kiválasztását teszi lehetővé. A panel a JPanel osztályból származik, így önálló komponensként jelenik meg a GUI-ban.

Adattagok:

- **JButton egyJatekos:** Egy játékos kiválasztásához.
- **JButton ketJatekos:** Két játékos kiválasztásához.
- **DifficultyPanel difficultyPanel:** Referencia a nehézségi szint panelhez.
- **StartingPlayerPanel startingPlayerPanel:** Referencia a kezdő játékos panelhez.
- **SymbolSelectionPanel symbolSelectionPanel:** Referencia a jel kiválasztó panelhez.

Metódusok:

- **PlayerNumberPanel():** Konstruktor
- **setReferencePanels(DifficultyPanel, StartingPlayerPanel, SymbolSelectionPanel):** void
- **updateGombok():** void

Eseménykezelők:

- **Egy_Click:** Az „1 játékos” gomb kiválasztását kezeli.
- **Ketto_Click:** A „2 játékos” gomb kiválasztását kezeli.

StartingPlayerPanel osztály

A StartingPlayerPanel osztály a beállítási ablak egyik alpanelje, amely a kezdő játékos kiválasztását teszi lehetővé egyjátékos módban. A panel a JPanel osztályból származik, így önálló komponensként jelenik meg a GUI-ban.

Adattagok:

- **JButton teKezd:** A felhasználó kezdését választó gomb.
- **JButton botKezd:** A bot kezdését választó gomb.

Metódusok:

- **StartingPlayerPanel():** Konstruktor
- **updateGombok():** void – a gombok vizuális állapotának frissítése a SettingsManager értékei alapján, engedélyezés és kiemelés szerint.

Eseménykezelők:

- **TeKezd_Click:** A „Én kezdek” gomb kiválasztását kezeli.
- **BotKezd_Click:** A „Bot kezd” gomb kiválasztását kezeli.

SymbolSelectionPanel osztály

A SymbolSelectionPanel osztály a beállítási ablak egyik alpanelje, amely lehetővé teszi a felhasználó jelének (X vagy O) kiválasztását egyjátékos módban. A panel a JPanel osztályból származik, így önálló komponensként jelenik meg a GUI-ban.

Adattagok:

- **JBUTTON xJel:** A „X” jel kiválasztásáért felelős gomb.
- **JBUTTON oJel:** Az „O” jel kiválasztásáért felelős gomb.

Metódusok:

- **SymbolSelectionPanel():** Konstruktor
- **updateGombok():** void – a gombok vizuális állapotának frissítése a SettingsManager értékei alapján, engedélyezés és kiemelés szerint.

Eseménykezelők:

- **X_Click:** Az „X” gomb kiválasztását kezeli.
- **O_Click:** Az „O” gomb kiválasztását kezeli.

SettingsManager osztály

A SettingsManager osztály egy statikus segédosztály, amely a játék beállításait tárolja. Az osztály minden adattagja statikus, így a teljes alkalmazásban központi helyen elérhető a beállítások állapota.

Adattagok:

- **boolean kezdoValasztva:** a „Könnyű” nehézségi szint kiválasztását jelzi (alapértelmezett: true).
- **boolean kozepesValasztva:** a „Közepes” nehézségi szint kiválasztását jelzi.
- **boolean nehezValasztva:** a „Nehéz” nehézségi szint kiválasztását jelzi.
- **boolean egyJatekosValasztva:** az 1 játékos mód választását jelzi (alapértelmezett: true).
- **boolean ketJatekosValasztva:** a 2 játékos mód választását jelzi.
- **boolean teKezdValasztva:** ha a felhasználó kezd, true; ha a bot kezd, false.
- **String felhasznaloJel:** a felhasználó jelét tárolja („X” vagy „O”, alapértelmezett: „X”).

Amoba osztály

Az Amoba osztály a játék fő logikáját és grafikus felületét valósítja meg. Ez az osztály felelős a 15×15-ös amőba tábla kirajzolásáért, a felhasználó és a bot/két játékos lépéseinek kezeléséért, valamint a győzelem és döntetlen ellenőrzéséért. Az osztály az ActionListener-t implementálja, így a gombokra történő kattintásokra reagál.

Adattagok:

- **Random random:** véletlenszerű kezdéshez és egyéb logikához.
- **JFrame keret:** a játék ablak.
- **JPanel szovegmezo, gombmezo:** a felső információs sáv és a tábla gombjai.
- **JButton[] gombok:** a 225 mezőből álló tábla gombjai.
- **JLabel szoveg:** a játék állapotát jelző szöveg (pl. „X jön”).
- **boolean jatekos_1:** jelzi, hogy X következik (true = X, false = O).
- **boolean jatekVege:** a játék véget ért-e.
- **JLabel balJatekos, jobbJatekos:** a játékosok neveit/jeleket mutató címkék.
- **Bot bot:** az egyjátékos mód esetén a bot kezelője.

Metódusok:

- **Amoba():** Konstruktor
- **Jatekosok():** beállítja a bal és jobb játékos feliratokat az aktuális játékbeállítások szerint (egy- vagy kétjátékos mód).
- **korok():** kezdeti lépés beállítása, véletlenszerűen vagy a beállítás szerint, bot lépésének indítása.
- **actionPerformed(ActionEvent e):** a játékos gombnyomásait kezeli, frissíti a mezőket és váltja a soron következő játékost, bot lépést indít.
- **ellenorzes():** ellenőrzi a győzelmi feltételeket sorok, oszlopok és átlók mentén, valamint a döntetlen állapotot.
- **dontetlen():** kezelni a döntetlent, színezéssel jelzi és visszatér a menübe 5 másodperc múlva.
- **X_gyozelem(int a, int b, int c, int d, int e):** X győzelmének kezelése, a nyertes gombok kiemelése, minden gomb lezárása és visszatérés a menübe.
- **O_gyozelem(int a, int b, int c, int d, int e):** O győzelmének kezelése, a nyertes gombok kiemelése, minden gomb lezárása és visszatérés a menübe.

Bot osztály

A Bot osztály az egyjátékos mód mesterséges intelligenciáját valósítja meg az amőba játékban. Ez az osztály dönt arról, hova lépjen a bot az aktuális táblaállás és a beállított nehézségi szint alapján.

Adattagok:

- **int intelligencia:** a bot nehézségi szintje (1 = kezdő, 2 = közepes, 3 = nehéz).
- **Random random:** véletlenszerű lépésekhez.
- **Amoba jatek:** a játék fő osztályának referenciaja, amin keresztül a bot lépéseket hajt végre.
- **String botJel:** a bot jele (X vagy O).
- **String ellenfelJel:** a játékos jele (X vagy O).

Metódusok:

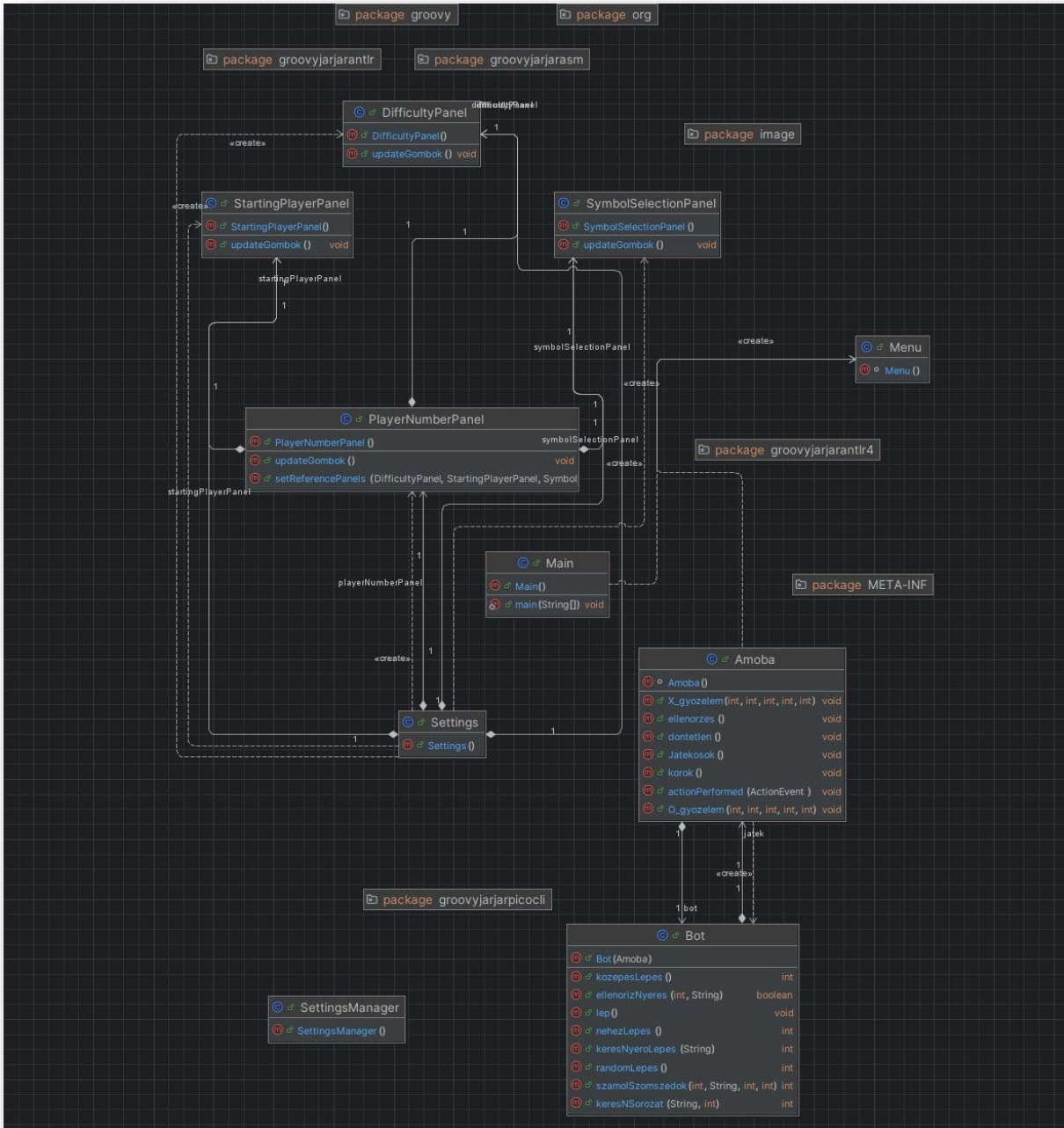
- **Bot(Amoba jatek):** Konstruktor
- **lep():** a bot aktuális lépését hajtja végre az intelligencia alapján, frissíti a táblát, a játékosváltást és ellenőrzi a győzelmet.
- **randomLepes():** véletlenszerűen választ egy üres mezőt.
- **kozepesLepes():** közepes szintű stratégia: nyerő lépés, blokkolás, középső mező, véletlenszerű lépés.
- **neheszLepes():** nehéz szintű stratégia: 4-es, 3-as, 2-es sorozatok kiterjesztése, ellenfél blokkolása, középső és stratégiai pozíciók, végül random lépés.
- **keresNyerolLepes(String jel):** keres egy lépést, ami azonnali győzelmet eredményez a megadott jel számára.
- **ellenorizNyeres(int index, String jel):** ellenőrzi, hogy az adott mező elhelyezése után 5 jel egymás mellett van-e.
- **keresNSorozat(String jel, int n):** keres n hosszú sorozatot, amit folytatni lehet.
- **szamolSzomszedok(int index, String jel, int sorIrany, int oszlopIrany):** megszámolja az adott jel szomszédos előfordulásait egy irányban maximum 4 mezőre.

Main osztály

A Main osztály a program belépési pontja.

UML DIAGRAM:

Az osztálydiagramot az IntelliJ IDEA beépített UML generáló eszközével készítettem, amely lehetővé tette, hogy a projekt osztályait, attribútumait, metódusait és azok kapcsolatait vizuálisan, szabványos UML jelöléssel ábrázoljam. A generált diagramot exportáltam, majd beillesztettem a dokumentumba, így szemléltetve a program felépítését és az osztályok közötti kapcsolatokat.



4. Megvalósítás

Az amőba játék megvalósítása objektumorientált megközelítéssel történt Java nyelven, Swing grafikus felülettel. A játék célja, hogy két játékos (vagy játékos és bot) 5 azonos jelet helyezzen el egymás mellett a 15×15-ös játéktáblán. A program moduláris felépítésű, minden funkció külön osztályban valósul meg, amely elősegíti a kód karbantarthatóságát és későbbi bővíthetőségét.

A játék fő komponensei: főmenü, beállítások, játéktábla, bot, játéklógika.

4.1 Osztályok bemutatása

Osztály	Rövid leírásokkal:
Menu	A program főmenüjét valósítja meg, ahol a felhasználó új játékot indíthat, a beállításokat módosíthatja, vagy kiléphet.
Settings	A játék paramétereinek konfigurálását kezeli.
DifficultyPanel	A nehézségi szint kiválasztását biztosítja.
PlayerNumberPanel	A játékosok számát kezeli (1 vagy 2 játékos).
StartingPlayerPanel	A kezdő játékos kiválasztását biztosítja.
SymbolSelectionPanel	A játékos jelének kiválasztását kezeli.
SettingsManager	Statikus osztály, a játék beállításait tárolja az egész alkalmazás számára.
Amoba	A játék fő logikáját és 15×15-ös tábláját kezeli. Feladata a játékoslépések kezelése, győzelem/döntetlen ellenőrzése és a GUI frissítése.
Bot	A bot intelligenciáját valósítja meg.
Main	A program belépési pontja.

- Minden panel (DifficultyPanel, PlayerNumberPanel, StartingPlayerPanel, SymbolSelectionPanel) a GUI külön moduljaként működik, így könnyen bővíthető és karbantartható.
- A SettingsManager statikus adatai biztosítják, hogy a beállítások minden osztályból elérhetők legyenek.
- A Bot osztály különböző nehézségi szinteket implementál, stratégiákkal a könnyű, közepes és nehéz játékhoz.

4.2 Osztályok főbb metódusainak bemutatása

Menu osztály

Konstruktor: Menu()

- Beállítja az ablak tulajdonságait (méret, középre igazítás, fix méret, bezárás kezelése).
- Betölti a háttérképet és elhelyezi a JLabel-t a menü címével.
- Létrehozza a gombokat (play, settings, quit), beállítja azok stílusát, méretét és elrendezését a gombPanel-en.
- Hozzáadja az eseménykezelőket a gombokhoz (Quit_Click, Settings_Click, Play_Click).
- A gombpanelt a háttérhez rendezi és megjeleníti az ablakot.

Eseménykezelő metódusok:

Quit_Click	Kilép a programból a System.exit(0) hívásával.
Settings_Click	Megnyitja a Settings ablakot.
Play_Click	Bezárja a menüt (dispose()) és elindítja az új játékot (new Amoba()).

Programkód:

```
// Eseménykezelők (actionok)
class Quit_Click implements ActionListener { 1 usage
    @Override
    public void actionPerformed(ActionEvent e) { System.exit( status: 0); }
}

class Settings_Click implements ActionListener { 1 usage
    @Override
    public void actionPerformed(ActionEvent e) { new Settings(); }
}

class Play_Click implements ActionListener { 1 usage
    @Override
    public void actionPerformed(ActionEvent e) {
        Menu.this.dispose();
        new Amoba();
    }
}
```

Settings osztály

Konstruktor: Settings()

- Beállítja az ablak tulajdonságait: cím (Beállítások), méret (500×400), középre igazítás, fix méret, bezárás kezelése (DISPOSE_ON_CLOSE).
- Betölti a háttérképet (/image/menu.png) és elhelyezi a JLabel-t az ablak tartalmaként.
- Létrehozza a felirat panelt, ami a „Beállítások” címet tartalmazza, és beállítja a betűtípust, színt.
- Létrehozza a fő panelt (mainPanel), amely 4 soros grid elrendezést használ a beállítási alpanelekhöz.
- Inicializálja a négy alpanelt:
 - **DifficultyPanel** – a nehézségi szint kiválasztásához
 - **PlayerNumberPanel** – az 1 vagy 2 játékos kiválasztásához
 - **StartingPlayerPanel** – a kezdő játékos kiválasztásához
 - **SymbolSelectionPanel** – a játékos jelének kiválasztásához (X/O)
- Beállítja a referencia panelek kapcsolatát a PlayerNumberPanel-ben, hogy az alpanelek tudják egymás állapotát.
- Hozzáadja az alpanelek komponenseit a fő panelhez, majd a fő panelt a háttér JLabel-hez.
- Frissíti az alpanelek gombjainak állapotát (updateGombok()), hogy a kezdeti beállítások megjelenjenek.
- Megjeleníti az ablakot (setVisible(true)).

DifficultyPanel osztály

Konstruktor: DifficultyPanel()

- Létrehozza a panelt a nehézségi szint kiválasztásához.
- Beállítja a panel átlátszóságát és elrendezését (FlowLayout balra).
- Létrehozza a "Könnyű", "Közepes" és "Nehéz" gombokat, beállítja azok stílusát, méretét és színét.
- Hozzáadja az eseménykezelőket a gombokhoz (Konnyu_Click, Kozepes_Click, Nehez_Click).
- A gombokat és a címkét hozzáadja a panelhez.

Metódus: updateGombok()

- Frissíti a gombok vizuális állapotát (engedélyezett/nem engedélyezett).
- Kiemeli a kiválasztott nehézségi szintet narancssárga színnel és bordázattal.
- Kétféleképpen letiltja a gombokat, jelezve, hogy nem választható a nehézségi szint.

Eseménykezelők:

- **Konnyu_Click** – A "Könnyű" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát és a gombok megjelenését.
- **Kozepes_Click** – A "Közepes" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát és a gombok megjelenését.
- **Nehez_Click** – A "Nehéz" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát és a gombok megjelenését.

Programkód:

```
public void updateGombok() { @usage
    boolean engedelyezett = !SettingsManager.ketJatekosValasztva;

    // Kattinthatóság beállítása
    kezdo.setEnabled(engedelyezett);
    kozepes.setEnabled(engedelyezett);
    nehez.setEnabled(engedelyezett);

    // Gombok beállítása
    if(engedelyezett) {
        kezdo.setForeground(SettingsManager.kezdoValasztva ? Color.ORANGE : Color.WHITE);
        kozepes.setForeground(SettingsManager.kozepesValasztva ? Color.ORANGE : Color.WHITE);
        nehez.setForeground(SettingsManager.nehezValasztva ? Color.ORANGE : Color.WHITE);

        kezdo.setBorder(new LineBorder(SettingsManager.kezdoValasztva ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
        kozepes.setBorder(new LineBorder(SettingsManager.kozepesValasztva ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
        nehez.setBorder(new LineBorder(SettingsManager.nehezValasztva ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
    } else {
        kezdo.setBorder(new LineBorder(new Color( r: 50, g: 50, b: 50), thickness: 2));
        kozepes.setBorder(new LineBorder(new Color( r: 50, g: 50, b: 50), thickness: 2));
        nehez.setBorder(new LineBorder(new Color( r: 50, g: 50, b: 50), thickness: 2));
    }
}

// Eseménykezelő az "könnyű" gombhoz
class Konnyu_Click implements ActionListener { @usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.kezdoValasztva = true;
        SettingsManager.kozepesValasztva = false;
        SettingsManager.nehezValasztva = false;
        updateGombok();
    }
}

// Eseménykezelő az "közepes" gombhoz
class Kozepes_Click implements ActionListener { @usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.kezdoValasztva = false;
        SettingsManager.kozepesValasztva = true;
        SettingsManager.nehezValasztva = false;
        updateGombok();
    }
}

// Eseménykezelő az "nehéz" gombhoz
class Nehez_Click implements ActionListener { @usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.kezdoValasztva = false;
        SettingsManager.kozepesValasztva = false;
        SettingsManager.nehezValasztva = true;
        updateGombok();
    }
}
```

PlayerNumberPanel osztály

Konstruktor: PlayerNumberPanel()

- Létrehozza a panelt a játékosok számának kiválasztásához.
- Beállítja a panel átlátszóságát és elrendezését (FlowLayout balra).
- Létrehozza az "1 játékos" és "2 játékos" gombokat, beállítja azok stílusát, méretét és színét.
- Hozzáadja az eseménykezelőket a gombokhoz (Egy_Click, Ketto_Click).
- A gombokat és a címkét hozzáadja a panelhez.

Metódus:

setReferencePanels(DifficultyPanel diffPanel, StartingPlayerPanel startPanel, SymbolSelectionPanel symbolPanel)

- Beállítja a panelhivatkozásokat a többi panelhez, hogy azok frissüljenek, amikor a játékosok számát módosítják.

updateGombok()

- Frissíti a gombok vizuális állapotát (kiemelés narancssárgával a kiválasztott opcióhoz).
- Biztosítja, hogy a panel mindig tükrözze a SettingsManager aktuális állapotát.

Eseménykezelők:

- **Egy_Click** – Az "1 játékos" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát, és frissíti az összes kapcsolódó panelt.
- **Ketto_Click** – A "2 játékos" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát, és frissíti az összes kapcsolódó panelt.

Programkód:

```
/**
 * Referencia beállítás a többi panelhez, hogy azok is frissüljenek
 * amikor a játékosok számát választjuk.
 */
public void setReferencePanels(DifficultyPanel diffPanel, StartingPlayerPanel startPanel, SymbolSelectionPanel symbolPanel) { 1 usage
    this.difficultyPanel = diffPanel;
    this.startingPlayerPanel = startPanel;
    this.symbolSelectionPanel = symbolPanel;
}

/**
 * Gombok állapotának frissítése a SettingsManager értékei alapján.
 * Kiemelés narancssárgára, ha éppen az adott opció van kiválasztva.
 */
public void updateGombok() { 3 usages
    egyJatekos.setForeground(SettingsManager.egyJatekosValasztva ? Color.ORANGE : Color.WHITE);
    ketJatekos.setForeground(SettingsManager.ketJatekosValasztva ? Color.ORANGE : Color.WHITE);

    egyJatekos.setBorder(new LineBorder(SettingsManager.egyJatekosValasztva ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
    ketJatekos.setBorder(new LineBorder(SettingsManager.ketJatekosValasztva ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
}

// Eseménykezelő az „1 játékos” gombhoz
class Egy_Click implements ActionListener { 1 usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.egyJatekosValasztva = true;
        SettingsManager.ketJatekosValasztva = false;
        updateGombok();
        if(difficultyPanel != null) difficultyPanel.updateGombok();
        if(startingPlayerPanel != null) startingPlayerPanel.updateGombok();
        if(symbolSelectionPanel != null) symbolSelectionPanel.updateGombok();
    }
}

// Eseménykezelő a „2 játékos” gombhoz
class Ketto_Click implements ActionListener { 1 usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.egyJatekosValasztva = false;
        SettingsManager.ketJatekosValasztva = true;
        updateGombok();
        if(difficultyPanel != null) difficultyPanel.updateGombok();
        if(startingPlayerPanel != null) startingPlayerPanel.updateGombok();
        if(symbolSelectionPanel != null) symbolSelectionPanel.updateGombok();
    }
}
```

StartingPlayerPanel osztály

Konstruktor: StartingPlayerPanel()

- Létrehozza a panelt a kezdő játékos kiválasztásához.
- Beállítja a panel átlátszóságát és elrendezését (FlowLayout balra).
- Létrehozza a "Én" és "Bot" gombokat, beállítja azok stílusát, méretét és színét.
- Hozzáadja az eseménykezelőket a gombokhoz (TeKezd_Click, BotKezd_Click).
- A gombokat és a címkét hozzáadja a panelhez.

Metódus: updateGombok()

- Frissíti a gombok vizuális állapotát a SettingsManager állapota alapján.
- Ha az 1 játékos mód nincs kiválasztva, a gombok letiltásra kerülnek.
- A kiválasztott opció narancssárga kiemelést kap.

Eseménykezelők:

- **TeKezd_Click** – Az "Én" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát és a gombok vizuális állapotát.
- **BotKezd_Click** – A "Bot" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát és a gombok vizuális állapotát.

Programkód:

```
/**
 * Gombok vizuális állapotának frissítése
 * - engedelyezett/new engedelyezett (pl. ha 1 játékos van, csak akkor lehet választani)
 * - kiválasztott gomb narancssárga kiemelést kap
 */

public void updateGombok() { 5 usages
    boolean engedelyezett = SettingsManager.egyJatekosValasztva;

    // Kattinthatóság beállítása
    teKezd.setEnabled(engedelyezett);
    botKezd.setEnabled(engedelyezett);

    // Gombok beállítása
    if(engedelyezett) {
        teKezd.setForeground(SettingsManager.teKezdValasztva ? Color.ORANGE : Color.WHITE);
        botKezd.setForeground(!SettingsManager.teKezdValasztva ? Color.ORANGE : Color.WHITE);

        teKezd.setBorder(new LineBorder(SettingsManager.teKezdValasztva ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
        botKezd.setBorder(new LineBorder(!SettingsManager.teKezdValasztva ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
    } else {
        teKezd.setBorder(new LineBorder(new Color( r: 50, g: 50, b: 50), thickness: 2));
        botKezd.setBorder(new LineBorder(new Color( r: 50, g: 50, b: 50), thickness: 2));
    }
}

// Eseménykezelő az „tekezd” gombhoz
class TeKezd_Click implements ActionListener { 1usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.teKezdValasztva = true;
        updateGombok();
    }
}

// Eseménykezelő az „botkezd” gombhoz
class BotKezd_Click implements ActionListener { 1usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.teKezdValasztva = false;
        updateGombok();
    }
}
```

SymbolSelectionPanel osztály

Konstruktor: SymbolSelectionPanel()

- Létrehozza a panelt a felhasználó jelének (X vagy O) kiválasztásához.
- Beállítja a panel átlátszóságát és elrendezését (FlowLayout balra).
- Létrehozza az "X" és "O" gombokat, beállítja azok stílusát, méretét és színét.
- Hozzáadja az eseménykezelőket a gombokhoz (X_Click, O_Click).
- A gombokat és a címkét hozzáadja a panelhez.

Metódus: updateGombok()

- Frissíti a gombok vizuális állapotát a SettingsManager értékei alapján.
- Ha két játékos van kiválasztva, a gombok letiltásra kerülnek.
- A kiválasztott jel narancssárga kiemelést kap.

Eseménykezelők:

- **X_Click** – Az "X" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát és a gombok vizuális állapotát.
- **O_Click** – Az "O" gomb kiválasztását kezeli; frissíti a SettingsManager állapotát és a gombok vizuális állapotát.

Programkód:

```
public void updateGombok() { 5 usages
    boolean engedelyezett = !SettingsManager.ketJatekosValasztva;

    xJel.setEnabled(engedelyezett);
    oJel.setEnabled(engedelyezett);

    if(engedelyezett) {
        xJel.setForeground(SettingsManager.felhasznaloJel.equals("X") ? Color.ORANGE : Color.WHITE);
        oJel.setForeground(SettingsManager.felhasznaloJel.equals("O") ? Color.ORANGE : Color.WHITE);

        xJel.setBorder(new LineBorder(SettingsManager.felhasznaloJel.equals("X") ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
        oJel.setBorder(new LineBorder(SettingsManager.felhasznaloJel.equals("O") ? Color.ORANGE : new Color( r: 30, g: 31, b: 34), thickness: 2));
    } else {
        xJel.setBorder(new LineBorder(new Color( r: 50, g: 50, b: 50), thickness: 2));
        oJel.setBorder(new LineBorder(new Color( r: 50, g: 50, b: 50), thickness: 2));
    }
}

// Eseménykezelő az „X” gombhoz
class X_Click implements ActionListener { 1 usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.felhasznaloJel = "X";
        updateGombok();
    }
}

// Eseménykezelő az „O” gombhoz
class O_Click implements ActionListener { 1 usage
    @Override
    public void actionPerformed(ActionEvent e) {
        SettingsManager.felhasznaloJel = "O";
        updateGombok();
    }
}
```

SettingsManager osztály

A SettingsManager egy statikus segédosztály, amely a játék beállításait tárolja. Nem hoz létre példányokat, hanem az összes adatot static mezőkben kezeli, így minden panel és komponens egyszerűen elérheti a játék beállításait.

Programkód:

```
public class SettingsManager { 56 usages

    // Nehézségi szint
    public static boolean kezdoValasztva = true; // kezdő szint alapértelmezett 6 usages
    public static boolean kozepesValasztva = false; 6 usages
    public static boolean nehezValasztva = false; 6 usages

    // Játékosok száma
    public static boolean egyJatekosValasztva = true; // 1 játékos alapértelmezett 9 usages
    public static boolean ketJatekosValasztva = false; 9 usages

    public static boolean teKezdValasztva = true; // felhasználó kezd 7 usages
    public static String felhasznaloJel = "X"; // felhasználó jele alapértelmezett X 13 usages
}
```

Amoba osztály

Konstruktor: Amoba()

- Létrehozza és beállítja a főablakot (JFrame) teljes képernyős módban.
- Inicializálja a táblagombokat (15×15) és a játékosok feliratait.
- Beállítja a szöveges információs feliratot (szoveg) és a bal/jobbs játékos jelzőket.
- Egyjátékos módban inicializálja a Bot objektumot.
- Meghívja a Jatekosok() és korok () metódusokat a játék indításához.

Főbb metódusok:

- **void Jatekosok()** – Beállítja a bal és jobb oldali feliratokat az egyjátékos vagy kétjátékos módnak megfelelően.
- **void korok()** – Inicializálja, ki kezd a játékban (felhasználó vagy bot, X vagy O), és kétjátékos módban véletlenszerűen választja a kezdőt.
- **void actionPerformed(ActionEvent e)** – Kezeli a játékos lépéseit, frissíti a táblát, és ha egyjátékos módban a bot a soron következő, elindítja a bot lépését késleltetéssel.
- **void ellenorzes()** – Ellenőrzi a táblát soronként, oszloponként és mindkét átló irányban, hogy van-e nyertes. Ha minden mező foglalt, döntetlen állapotba kerül.
- **void dontetlen()** – Döntetlen esetén kiírja az üzenetet, sárgára színezi a mezőket és 5 másodperc után visszatér a menübe.
- **void X_gyozelem(int a, int b, int c, int d, int e)** – X győzelmének kezelése: a győztes mezőket zöldre színezi, tiltja a további lépéseket, 5 másodperc múlva visszatér a menübe.
- **void O_gyozelem(int a, int b, int c, int d, int e)** – O győzelmének kezelése: a győztes mezőket zöldre színezi, tiltja a további lépéseket, 5 másodperc múlva visszatér a menübe.

Programkód:

```
@Override
public void actionPerformed(ActionEvent e) {
    if (jatekVege) return;
    for(int i=0; i<225; i++) {
        if(e.getSource()==gombok[i]) {
            // Üres mező ellenőrzése
            if(gombok[i].getText().equals("")) {

                // Egyjátékos mód
                if(SettingsManager.egyJatekosValasztva) {
                    // Csak akkor léphet a játékos, ha ő van soron
                    if((jatekos_1 && SettingsManager.felhasznaloJel.equals("X")) ||
                        (!jatekos_1 && SettingsManager.felhasznaloJel.equals("0"))) {

                        // Felhasználó lépése
                        if(SettingsManager.felhasznaloJel.equals("X")) {
                            gombok[i].setForeground(new Color( r: 255, g: 0, b: 0));
                            gombok[i].setText("X");
                            jatekos_1 = false;
                            if (!jatekVege) szoveg.setText("0 jön");
                        } else {
                            gombok[i].setForeground(new Color( r: 0, g: 0, b: 255));
                            gombok[i].setText("0");
                            jatekos_1 = true;
                            if (!jatekVege) szoveg.setText("X jön");
                        }
                    }

                    ellenorzes();

                    // Bot lépése késleltetéssel
                    Timer t = new Timer( delay: 300, new ActionListener() {
                        @Override
                        public void actionPerformed(ActionEvent e) { if (!jatekVege) bot.lep(); }
                    });
                    t.setRepeats(false);
                    t.start();
                }
            }

            // Kétijátékos mód
            else if(SettingsManager.ketJatekosValasztva) {
                if(jatekos_1) {
                    gombok[i].setForeground(new Color( r: 255, g: 0, b: 0));
                    gombok[i].setText("X");
                    jatekos_1 = false;
                    if (!jatekVege) szoveg.setText("0 jön");
                    ellenorzes();
                } else {
                    gombok[i].setForeground(new Color( r: 0, g: 0, b: 255));
                    gombok[i].setText("0");
                    jatekos_1 = true;
                    if (!jatekVege) szoveg.setText("X jön");
                    ellenorzes();
                }
            }
        }
    }
}
```

```

// Játékosok mutatása
public void Jatekosok() { 1usage
    if (SettingsManager.egyJatekosValasztva) {
        // Egyjátékos mód
        if (SettingsManager.felhasznaloJel.equals("X")) {
            balJatekos.setText("Te (X)");
            jobbJatekos.setText("Bot (0)");
        } else {
            balJatekos.setText("Te (0)");
            jobbJatekos.setText("Bot (X)");
        }
    } else if (SettingsManager.ketJatekosValasztva) {
        // Kétjátékos mód
        balJatekos.setText("Játékos 1 (X)");
        jobbJatekos.setText("Játékos 2 (0)");
    }
}

public void korok() { 1usage

    try {
        Thread.sleep( millis: 2000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    // Kétjátékos mód: véletlenszerűen kezd X vagy 0
    if (SettingsManager.ketJatekosValasztva) {
        if (random.nextInt( bound: 2) == 0) {
            jatekos_1 = true;
            szoveg.setText("X jön");
        } else {
            jatekos_1 = false;
            szoveg.setText("0 jön");
        }
    }

    // Egyjátékos mód: beállítjuk, ki kezd, felhasználó vagy bot
    else if (SettingsManager.egyJatekosValasztva) {
        // Ha a felhasználó kezd
        if (SettingsManager.teKezdValasztva) {
            // Felhasználó jele alapján állítjuk be
            if (SettingsManager.felhasznaloJel.equals("X")) {
                jatekos_1 = true; // Felhasználó (X) kezd
                szoveg.setText("X jön");
            } else {
                jatekos_1 = false; // Felhasználó (0) kezd
                szoveg.setText("0 jön");
            }
        }

        // Ha a bot kezd
        else {
            if (SettingsManager.felhasznaloJel.equals("X")) {
                jatekos_1 = false; // Bot (0) kezd
                szoveg.setText("0 jön");
                bot.lep(); // Bot lép
            } else {
                jatekos_1 = true; // Bot (X) kezd
                szoveg.setText("X jön");
                bot.lep(); // Bot lép
            }
        }
    }
}

```

```

// Ellenőrzi a győzelmi feltételeket minden irányban
public void ellenorzes() { 4 usages
    int sor_max = 15;
    int oszlop_max = 15;

    // soronkénti ellenőrzés
    // Ellenőrizzük a sorokat 5 azonos jel után
    for(int sor=0; sor < sor_max; sor++) {
        for(int oszlop=0; oszlop <= oszlop_max - 5; oszlop++) {
            int idx = sor * 15 + oszlop;
            String jel = gombok[idx].getText();
            if (!jel.equals("") &&
                jel.equals(gombok[idx + 1].getText()) &&
                jel.equals(gombok[idx + 2].getText()) &&
                jel.equals(gombok[idx + 3].getText()) &&
                jel.equals(gombok[idx + 4].getText())) {

                if (jel.equals("X")) {
                    X_gyozelem(idx, b: idx + 1, c: idx + 2, d: idx + 3, e: idx + 4);
                } else if (jel.equals("0")) {
                    O_gyozelem(idx, b: idx + 1, c: idx + 2, d: idx + 3, e: idx + 4);
                }
                return;
            }
        }
    }
}

// oszloponkénti ellenőrzés
// Ellenőrizzük az oszlopokat 5 azonos jel után
for(int oszlop = 0; oszlop < oszlop_max; oszlop++) {
    for(int sor = 0; sor <= sor_max - 5; sor++) {
        int idx = sor * 15 + oszlop;
        String jel = gombok[idx].getText();
        if (!jel.equals("") &&
            jel.equals(gombok[idx + 15].getText()) &&
            jel.equals(gombok[idx + 30].getText()) &&
            jel.equals(gombok[idx + 45].getText()) &&
            jel.equals(gombok[idx + 60].getText())) {

            if (jel.equals("X")) {
                X_gyozelem(idx, b: idx + 15, c: idx + 30, d: idx + 45, e: idx + 60);
            } else if (jel.equals("0")) {
                O_gyozelem(idx, b: idx + 15, c: idx + 30, d: idx + 45, e: idx + 60);
            }
            return;
        }
    }
}
}

```

```

// átlós ellenőrzés (\ irány)
// Ellenőrizzük a diagonális (\) irányt
for(int sor = 0; sor <= sor_max - 5; sor++) {
    for(int oszlop = 0; oszlop <= oszlop_max - 5; oszlop++) {
        int idx = sor * 15 + oszlop;
        String jel = gombok[idx].getText();
        if (!jel.equals("")) &&
            jel.equals(gombok[idx + 16].getText()) && // 15 + 1
            jel.equals(gombok[idx + 32].getText()) && // 15*2 + 2
            jel.equals(gombok[idx + 48].getText()) && // 15*3 + 3
            jel.equals(gombok[idx + 64].getText())) { // 15*4 + 4

            if (jel.equals("X")) {
                X_gyozelem(idx, b idx + 16, c idx + 32, d idx + 48, e idx + 64);
            } else if (jel.equals("0")) {
                O_gyozelem(idx, b idx + 16, c idx + 32, d idx + 48, e idx + 64);
            }
            return;
        }
    }
}

// átlós ellenőrzés (/ irány)
// Ellenőrizzük a diagonális (/) irányt
for(int sor = 0; sor <= sor_max - 5; sor++) {
    for(int oszlop = 4; oszlop < oszlop_max; oszlop++) { // oszlop >= 4, hogy ne menjünk negatívba
        int idx = sor * 15 + oszlop;
        String jel = gombok[idx].getText();
        if (!jel.equals("")) &&
            jel.equals(gombok[idx + 14].getText()) && // 15 - 1
            jel.equals(gombok[idx + 28].getText()) && // 15*2 - 2
            jel.equals(gombok[idx + 42].getText()) && // 15*3 - 3
            jel.equals(gombok[idx + 56].getText())) { // 15*4 - 4

            if (jel.equals("X")) {
                X_gyozelem(idx, b idx + 14, c idx + 28, d idx + 42, e idx + 56);
            } else if (jel.equals("0")) {
                O_gyozelem(idx, b idx + 14, c idx + 28, d idx + 42, e idx + 56);
            }
            return;
        }
    }
}

// Döntetlen esetén
boolean vanUresMezo = false;
for(int i = 0; i < 225; i++) {
    if(gombok[i].getText().equals("")) {
        vanUresMezo = true;
        break;
    }
}

// Ha nincs üres mező, döntetlen
if(!vanUresMezo) {
    dontetlen();
}
}

```

```

// Döntetlen vizsgálata
public void dontetlen() { 1 usage
    if (jatekVege) return;
    jatekVege = true;

    // Minden gombot sárgára színezzünk
    for(int i = 0; i < 225; i++) {
        gombok[i].setBackground(Color.YELLOW);
        gombok[i].setEnabled(false);
    }

    szoveg.setText("Döntetlen!");

    // 5 másodperc múlva visszatérés a menübe
    Timer timer = new Timer(delay: 5000, new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            keret.dispose();
            new Menu();
        }
    });
    timer.setRepeats(false);
    timer.start();
}

// X jel győzelének beállítása
public void X_gyozelen(int a, int b, int c, int d, int e) { 4 usages
    if (jatekVege) return;
    jatekVege = true;

    gombok[a].setBackground(Color.GREEN);
    gombok[b].setBackground(Color.GREEN);
    gombok[c].setBackground(Color.GREEN);
    gombok[d].setBackground(Color.GREEN);
    gombok[e].setBackground(Color.GREEN);
    for(int i=0; i<225; i++) {
        gombok[i].setEnabled(false);
    }
    szoveg.setText("X nyert!");

    Timer timer = new Timer(delay: 5000, new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            keret.dispose(); // bezárjuk az Amőba ablakot
            new Menu();      // visszatérés a menübe
        }
    });
    timer.setRepeats(false); // egyszeri futtatás
    timer.start();
}

```

Bot osztály

Konstruktor: Bot(Amoba jatek)

- Beállítja a hivatkozást a játékra (this.jatek = jatek)
- Beállítja az **intelligencia szintet** a SettingsManager alapján:
 - kezdőValasztva → 1
 - közepesValasztva → 2
 - nehézValasztva → 3
- Beállítja a **bot és ellenfél jeleit** a SettingsManager.felhasznaloJel alapján:
 - Ha felhasználó "X" → bot = "O", ellenfél = "X"
 - Ha felhasználó "O" → bot = "X", ellenfél = "O"

Metódusok

lep()

- Meghatározza a bot aktuális lépését az intelligencia alapján:
 - 1 → randomLepes()
 - 2 → kozepesLepes()
 - 3 → nehézLepes()
- Ha van érvényes lépés:
 - Beállítja a gomb szövegét (botJel) és színét
 - Váltja a játékosok körét (jatekos_1)
 - Frissíti a státusz szöveget (szoveg)
 - Ellenőrzi a győzelmet (ellenorzes())

randomLepes()

- Visszaad egy véletlenszerű üres mező indexét
- Ha nincs üres mező → -1

kozepesLepes()

- Ellenőrzi a **nyerő lépést** (keresNyeroLepes(botJel))
- Blokkolja az ellenfél nyerő lépését (keresNyeroLepes(ellenfelJel))
- Ha a középső mező szabad → oda lép
- Ha egyik sem → véletlenszerű lépés

nehézLepes()

- Stratégiai, többlépcsős döntés:
 1. Saját 4-es sorozat befejezése (keresNSorozat(botJel, 4))
 2. Ellenfél 4-es blokkolása
 3. Saját 3-as folytatása
 4. Ellenfél 3-as blokkolása
 5. Saját 2-es folytatása
 6. Ellenfél 2-es blokkolása
 7. Középső mező foglalása
 8. Stratégiai pozíciók (pl. közép körüli mezők)
 9. Véletlenszerű lépés

keresNyeroLepes(String jel)

- Végigmegy az összes üres mezőn
- Szimulálja a lépést
- Ellenőrzi, hogy az adott jel győzelmet eredményez-e (ellenorizNyeres)
- Ha talál → visszaadja az indexet
- Ha nem → -1

ellenorizNyeres(int index, String jel)

- Ellenőrzi a győzelmet **vízszintesen, függőlegesen, átlóban** (\ és /)
- Ha 5 azonos jel egymás mellett → true
- Egyébként → false

keresNSorozat(String jel, int n)

- Keres olyan üres mezőt, ahol a jel n hosszú sorozatot tud folytatni
- Ellenőrzi minden irányban (vízszintes, függőleges, átlók)
- Ha talál → visszaadja az indexet
- Ha nem → -1

szamolSzomszedok(int index, String jel, int sorIrtany, int oszlopIrtany)

- Megszámolja az adott jel szomszédos előfordulásait adott irányban (max 4 mező)

Programkód:

```
// a bot aktuális lépését végrehajtja intelligencia alapján
public void lep() { 3 usages
    int index = -1;

    if (intelligencia == 1) {
        index = randomLepes();
    } else if (intelligencia == 2) {
        index = kozepesLepes();
    } else if (intelligencia == 3) {
        index = nehezLepes();
    }

    // Ha van érvényes lépés
    if (index != -1) {
        if (botJel.equals("X")) {
            jatek.gombok[index].setForeground(new java.awt.Color(0, 255, 0));
        } else {
            jatek.gombok[index].setForeground(new java.awt.Color(255, 0, 0));
        }
        jatek.gombok[index].setText(botJel);
        jatek.jatekos_1 = !jatek.jatekos_1;
        jatek.szoveg.setText(ellenfelJel + " jön");
        jatek.ellenorzes();
    }
}

// Randomlépés (véletlenszerű lépés) beállítása
public int randomLepes() { 3 usages
    List<Integer> uresMezok = new ArrayList<>();
    for (int i = 0; i < jatek.gombok.length; i++) {
        if (jatek.gombok[i].getText().equals("")) {
            uresMezok.add(i);
        }
    }
    if (!uresMezok.isEmpty()) {
        return uresMezok.get(random.nextInt(uresMezok.size()));
    }
    return -1;
}
```

```

public int kozepesLepes() { ! usage
    // 1. Megpróbálunk nyerni
    int nyeroLepes = keresNyeroLepes(botJel);
    if (nyeroLepes != -1) return nyeroLepes;

    // 2. Blokkoljuk az ellenfelet
    int blokkLepes = keresNyeroLepes(ellenfelJel);
    if (blokkLepes != -1) return blokkLepes;

    // 3. Középső mező előnyben
    int kozepso = 112;
    if (jatek.gombok[kozepso].getText().equals("")) {
        return kozepso;
    }

    // 4. Random lépés
    return randomLepes();
}

// Nehéz szintű lépés
public int nehézLepes() { ! usage
    // 1. Nyero lépés keresése (4-es sorok)
    int nyero4 = keresNSorozat(botJel, 4);
    if (nyero4 != -1) return nyero4;

    // 2. Ellenfél 4-es blokkolása
    int blokk4 = keresNSorozat(ellenfelJel, 4);
    if (blokk4 != -1) return blokk4;

    // 3. Saját 3-as folytatása
    int nyero3 = keresNSorozat(botJel, 3);
    if (nyero3 != -1) return nyero3;

    // 4. Ellenfél 3-as blokkolása
    int blokk3 = keresNSorozat(ellenfelJel, 3);
    if (blokk3 != -1) return blokk3;

    // 5. Saját 2-es folytatása
    int nyero2 = keresNSorozat(botJel, 2);
    if (nyero2 != -1) return nyero2;

    // 6. Ellenfél 2-es blokkolása
    int blokk2 = keresNSorozat(ellenfelJel, 2);
    if (blokk2 != -1) return blokk2;

    // 7. Középső mező
    int kozepso = 112;
    if (jatek.gombok[kozepso].getText().equals("")) {
        return kozepso;
    }

    // 8. Stratégiai pozíciók (közép körüli mezők)
    int[] strategiai = {97, 98, 99, 111, 113, 126, 127, 128};
    for (int idx : strategiai) {
        if (jatek.gombok[idx].getText().equals("")) {
            return idx;
        }
    }

    // 9. Random lépés
    return randomLepes();
}

```

```

// Keres egy lépést, ami azonnali győzelmet eredményez a megadott jel számára.
public int keresNyeroLepes(String jel) { 2 usages
    // Keresünk egy lépést, ami 5-öt eredményez
    for (int i = 0; i < jatek.gombok.length; i++) {
        if (jatek.gombok[i].getText().equals("")) {
            // Szimuláljuk a lépést
            jatek.gombok[i].setText(jel);
            boolean nyer = ellenorizNyeres(i, jel);
            jatek.gombok[i].setText(""); // visszaállítás

            if (nyer) return i;
        }
    }
    return -1;
}

// Ellenőrzi, hogy az adott mezőn a jel 5-öt eredményez-e.
public boolean ellenorizNyeres(int index, String jel) { 1 usages
    int sor = index / 15;
    int oszlop = index % 15;

    // Vízszintes ellenőrzés
    int db = 1;
    for (int i = oszlop - 1; i >= 0 && jatek.gombok[sor * 15 + i].getText().equals(jel); i--) db++;
    for (int i = oszlop + 1; i < 15 && jatek.gombok[sor * 15 + i].getText().equals(jel); i++) db++;
    if (db >= 5) return true;

    // Függőleges
    db = 1;
    for (int i = sor - 1; i >= 0 && jatek.gombok[i * 15 + oszlop].getText().equals(jel); i--) db++;
    for (int i = sor + 1; i < 15 && jatek.gombok[i * 15 + oszlop].getText().equals(jel); i++) db++;
    if (db >= 5) return true;

    // Átló \
    db = 1;
    for (int i = 1; sor - i >= 0 && oszlop - i >= 0 && jatek.gombok[(sor - i) * 15 + (oszlop - i)].getText().equals(jel); i++) db++;
    for (int i = 1; sor + i < 15 && oszlop + i < 15 && jatek.gombok[(sor + i) * 15 + (oszlop + i)].getText().equals(jel); i++) db++;
    if (db >= 5) return true;

    // Átló /
    db = 1;
    for (int i = 1; sor - i >= 0 && oszlop + i < 15 && jatek.gombok[(sor - i) * 15 + (oszlop + i)].getText().equals(jel); i++) db++;
    for (int i = 1; sor + i < 15 && oszlop - i >= 0 && jatek.gombok[(sor + i) * 15 + (oszlop - i)].getText().equals(jel); i++) db++;
    if (db >= 5) return true;

    return false;
}

```

```

// Keres n hosszú sorozatot, amit folytatni lehet.
public int keresSorozat(String jel, int n) { 6 usages
    // Keresünk n hosszú sorozatot, amit folytatni lehet
    for (int i = 0; i < jatek.gombok.length; i++) {
        if (jatek.gombok[i].getText().equals("")) {
            int sor = i / 15;
            int oszlop = i % 15;

            // Vízszintes
            int db = szamolSzomszedok(i, jel, sorIrány: 0, oszlopIrány: 1) + szamolSzomszedok(i, jel, sorIrány: 0, oszlopIrány: -1);
            if (db >= n) return i;

            // Függőleges
            db = szamolSzomszedok(i, jel, sorIrány: 1, oszlopIrány: 0) + szamolSzomszedok(i, jel, sorIrány: -1, oszlopIrány: 0);
            if (db >= n) return i;

            // Átló \
            db = szamolSzomszedok(i, jel, sorIrány: 1, oszlopIrány: 1) + szamolSzomszedok(i, jel, sorIrány: -1, oszlopIrány: -1);
            if (db >= n) return i;

            // Átló /
            db = szamolSzomszedok(i, jel, sorIrány: 1, oszlopIrány: -1) + szamolSzomszedok(i, jel, sorIrány: -1, oszlopIrány: 1);
            if (db >= n) return i;
        }
    }
    return -1;
}

// Megszámolja az adott jel szomszédos előfordulásait egy irányban maximum 4 mezőre.
public int szamolSzomszedok(int index, String jel, int sorIrány, int oszlopIrány) { 8 usages
    int sor = index / 15;
    int oszlop = index % 15;
    int db = 0;

    for (int i = 1; i < 5; i++) {
        int ujSor = sor + (sorIrány * i);
        int ujOszlop = oszlop + (oszlopIrány * i);

        if (ujSor < 0 || ujSor >= 15 || ujOszlop < 0 || ujOszlop >= 15) break;

        int idx = ujSor * 15 + ujOszlop;
        if (jatek.gombok[idx].getText().equals(jel)) {
            db++;
        } else {
            break;
        }
    }
    return db;
}

```

Main osztály

Ez a Main osztály **indító pontja** a programnak.

Röviden:

- `public static void main(String[] args)` → a program belépési pontja.
- `Menu menu = new Menu();` → létrehozza és megjeleníti a fő menüt.

Programkód:

```
public class Main {  
    public static void main(String[] args) {  
  
        Menu menu = new Menu();  
  
    }  
}
```

4.3 Fájlkezelés

```
// Mentés fájlba  
public static void ment() { no usages  
    try (BufferedWriter writer = new BufferedWriter(new FileWriter("settings.txt"))) {  
        writer.write(str: egyJatekosValasztva + "\n");  
        writer.write(str: ketJatekosValasztva + "\n");  
        writer.write(str: teKezdValasztva + "\n");  
        writer.write(str: kezdoValasztva + "\n");  
        writer.write(str: kozepesValasztva + "\n");  
        writer.write(str: nehezValasztva + "\n");  
        writer.write(str: felhasznaloJel + "\n");  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}  
  
// Beolvasás fájlból  
public static void betolt() { no usages  
    try (BufferedReader reader = new BufferedReader(new FileReader("settings.txt"))) {  
        egyJatekosValasztva = Boolean.parseBoolean(reader.readLine());  
        ketJatekosValasztva = Boolean.parseBoolean(reader.readLine());  
        teKezdValasztva = Boolean.parseBoolean(reader.readLine());  
        kezdoValasztva = Boolean.parseBoolean(reader.readLine());  
        kozepesValasztva = Boolean.parseBoolean(reader.readLine());  
        nehezValasztva = Boolean.parseBoolean(reader.readLine());  
        felhasznaloJel = reader.readLine();  
    } catch (IOException e) {  
        System.out.println("Settings.txt nem található");  
    }  
}
```

5. Tesztelés

A teszteléshez létrehoztam egy test nevű mappát, amelyben a Tesztek.java fájl található. A tesztek lefedik a SettingsManager, a Bot és az Amoba osztályok működését, ellenőrizve a különböző beállításokat, a bot lépéseit, valamint a játék állapotának és győzelmi feltételeinek helyes kezelését.

```
// SettingsManager tesztek

@Test
void testDefaultDifficulty() {
    // Ellenőrizzük, hogy a kezdő nehézség alapértelmezett-e
    assertTrue(SettingsManager.kezdoValasztva);
    assertFalse(SettingsManager.kozepesValasztva);
    assertFalse(SettingsManager.nehezValasztva);
}

@Test
void testSetMediumDifficulty() {
    // Beállítjuk a közepes nehézséget és ellenőrizzük
    SettingsManager.kezdoValasztva = false;
    SettingsManager.kozepesValasztva = true;
    assertTrue(SettingsManager.kozepesValasztva);
    assertFalse(SettingsManager.kezdoValasztva);
    assertFalse(SettingsManager.nehezValasztva);
}

@Test
void testSymbolChange() {
    // Felhasználói jel változtatásának tesztelése
    SettingsManager.felhasznaloJel = "0";
    assertEquals( expected: "0", SettingsManager.felhasznaloJel);
}
```

```

// Bot tesztek
@Test
void testRandomMove() {
    SettingsManager.kezdoValasztva = true;
    SettingsManager.kozepesValasztva = false;
    SettingsManager.nehezValasztva = false;
    bot = new Bot(game);
    // Véletlenszerű lépés generálása valid indexre
    int idx = bot.randomLepes();
    assertTrue( condition: idx >= 0 && idx < 225);
}

@Test
void testMediumMove() {
    SettingsManager.kezdoValasztva = false;
    SettingsManager.kozepesValasztva = true;
    SettingsManager.nehezValasztva = false;
    bot = new Bot(game);

    // Közepes nehézségű lépés valid indexre
    int idx = bot.kozepesLepes();
    assertTrue( condition: idx >= 0 && idx < 225);
}

@Test
void testHardMove() {
    // Nehéz szint beállítása
    SettingsManager.kezdoValasztva = false;
    SettingsManager.kozepesValasztva = false;
    SettingsManager.nehezValasztva = true;
    bot = new Bot(game);

    // Nehéz szintű lépés valid indexre
    int idx = bot.nehezLepes();
    assertTrue( condition: idx >= 0 && idx < 225);
}

```

```
// Amoba tesztek
```

```
@Test
```

```
void testDefaultStartingPlayer() {
```

```
    // Alapértelmezett, felhasználó kezd X-szel
```

```
    game.korok();
```

```
    // Ellenőrizzük a korok metódust
```

```
    if (SettingsManager.felhasznaloJel.equals("X") && SettingsManager.teKezdValasztva) {
```

```
        assertTrue(game.jatekos_1);
```

```
    } else if (SettingsManager.felhasznaloJel.equals("0") && SettingsManager.teKezdValasztva) {
```

```
        assertFalse(game.jatekos_1);
```

```
    }
```

```
}
```

```
@Test
```

```
void testXWinDetection() {
```

```
    // Tegyük 5 X-et vízszintesen az első sorba
```

```
    game.gombok[0].setText("X");
```

```
    game.gombok[1].setText("X");
```

```
    game.gombok[2].setText("X");
```

```
    game.gombok[3].setText("X");
```

```
    game.gombok[4].setText("X");
```

```
    // Ellenőrizzük, hogy a játék véget ért
```

```
    game.ellenorzes();
```

```
    assertTrue(game.jatekVege); // X győzelem miatt vége kell legyen
```

```
}
```

6. Felhasználói kézikönyv

Főmenü

A program indításakor egy menü jelenik meg, amely három menüpontot tartalmaz:

- **Beállítások:** a játék paramétereinek testreszabása
- **Játék (új játék):** új játék indítása
- **Kilépés:** a játékból való kilépés

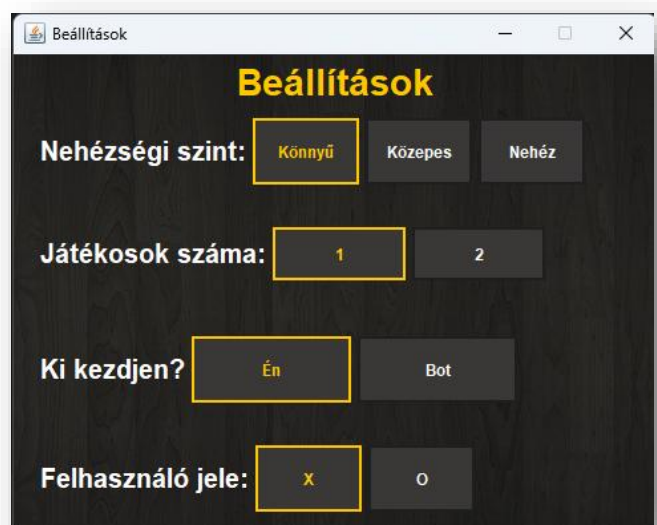


1. A "Beállítások" gombra kattintva a játék paramétereit lehet megadni. A beállítások attól függően változnak, hogy egy- vagy kétjátékos módot választunk.

1.1 Egyjátékos mód (1 játékos)

Ebben a módban a játékos egy bot ellen játszik. Beállítható:

- **Nehézségi szint:**
 - Könnyű
 - Közepes
 - Nehéz
- **Ki kezdje a játékot:**
 - Játékos
 - Bot
- **A játékos szimbóluma:**
 - X
 - O



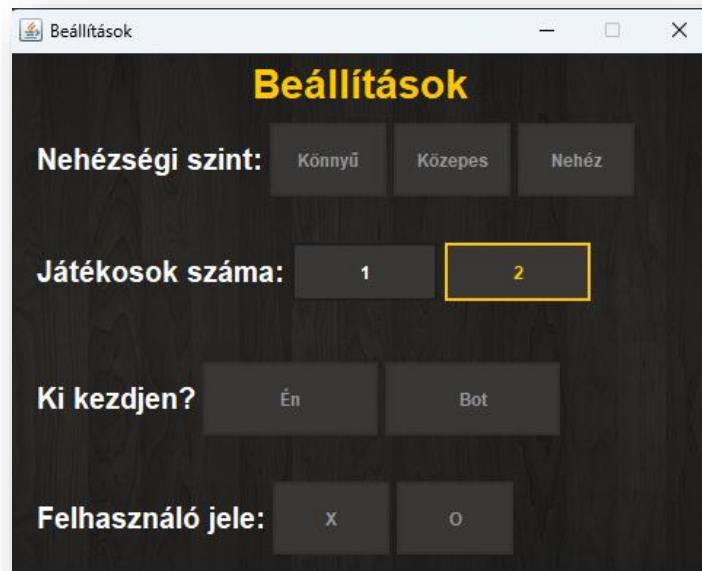
1. ábra az alapértelmezett beállítások

1.2 Kétjátékos mód (2 játékos)

Ebben a módban **nem állítható**:

- Nehézségi szint
- Ki kezdjen
- A játékos szimbóluma

Ezek automatikusan beállításra kerülnek a kétjátékos játékmenethez.



A beállítások az **ablak bezárásakor (X gomb)** automatikusan elmentődnek, és a program visszatér a főmenübe. **Fontos:** A beállítások nem véglegesen tárolódnak. A program újraindításakor a beállítások visszaállnak az alapértelmezettre.



2. A „Kilépés” gombra kattintva a program azonnal bezárul.

```
Process finished with exit code 0
```

3. Az új játék gombra kattintva:

A „Játék (Új játék)” gombra kattintva megjelenik a **15×15-ös játéktábla**.

A képernyő tetején megjelenik:

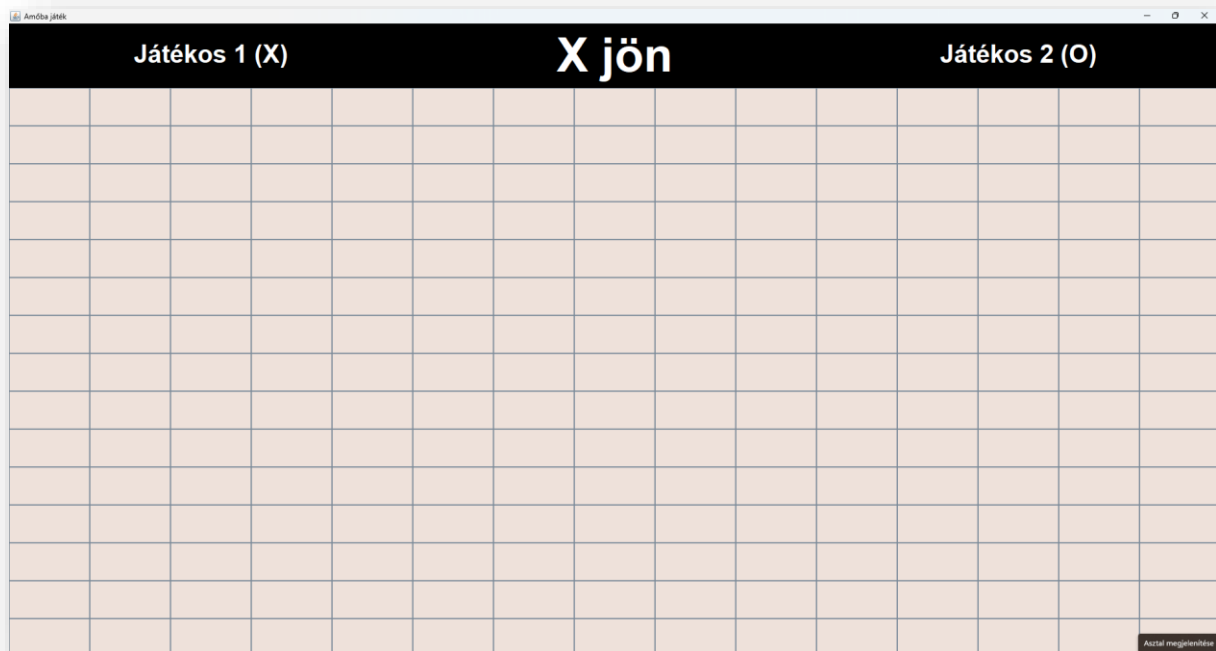
- melyik játékos milyen szimbólummal játszik (X vagy O),
- ki kezdi a játszmát.

A játék során a mezőkre kattintva lehet elhelyezni a jeleket.

Egyjátékos mód



Kétjátékos mód



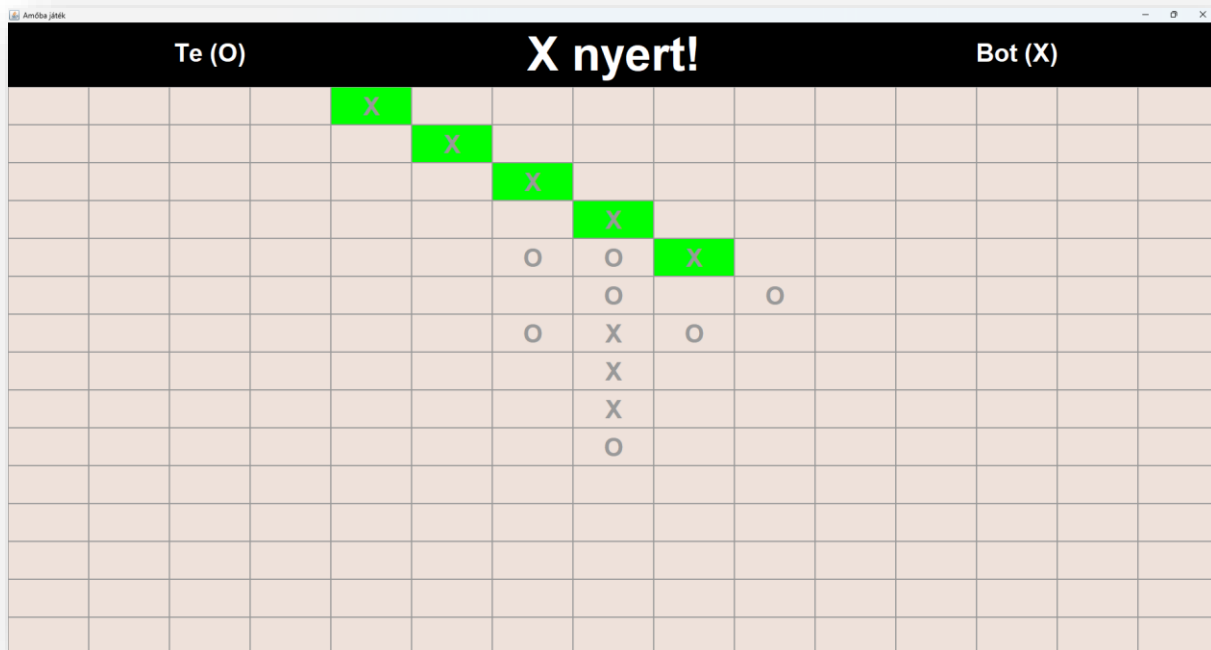
Győzelem

A játék akkor ér véget, amikor valamelyik játékos **5 azonos jelet helyez el egy sorban**, akár:

- vízszintesen,
- függőlegesen,
- átlósan.

Ilyenkor:

- a győztes 5 mező zöld háttérrel kiemelődik,
- a többi mező elszürkül,
- a felső szöveg jelzi a győztest,
- majd néhány másodperc múlva a program automatikusan visszatér a főmenübe.

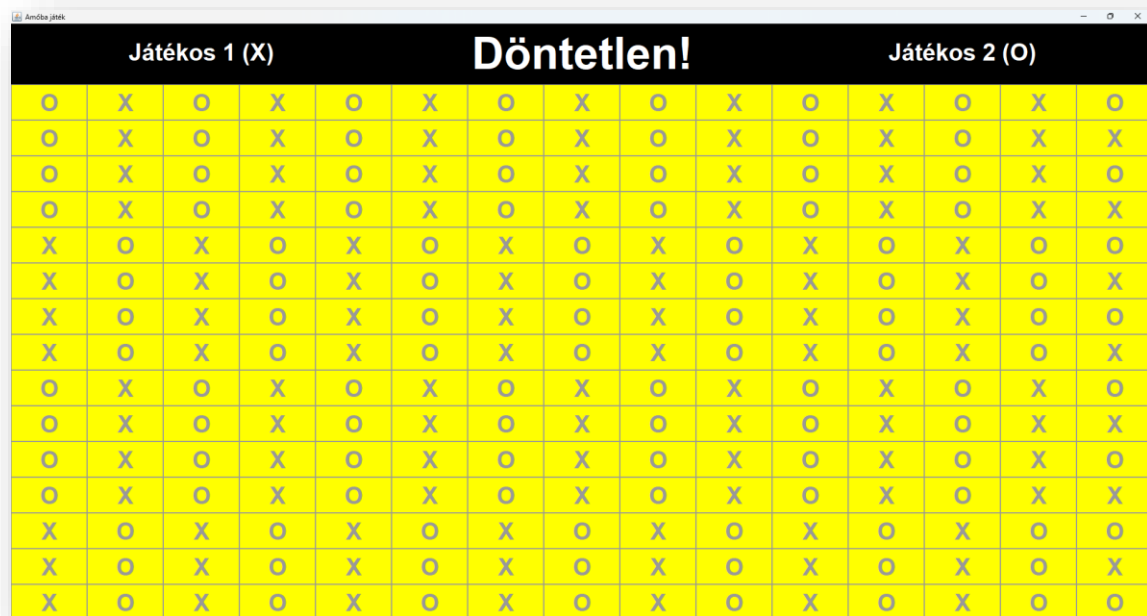


Döntetlen akkor következik be, ha:

- a tábla megtelik,
- és egyik játékos sem tudott 5 jelet kirakni.

Ilyenkor:

- a pálya besárgul,
- a felső szöveg „Döntetlen!” feliratot jelenít meg,
- majd a program visszatér a főmenübe.



7. Források

BME – Programozás alapjai 3 (előadások és laborok)

Szoftverfejlesztés JAVA SE platformon

W3School: <https://www.w3schools.com/java/>

Stack Overflow: <https://stackoverflow.com/questions>

YouTube: Java GUI tutorialok (Swing, JavaFX)

<https://www.youtube.com/@BroCodez>

<https://www.youtube.com/@KennyYipCoding>